2026 IEEE International Conference on Optics, Machine Learning and Emerging Technology (OMLET)
29 – 31 October 2026, Nairobi, Kenya

Multi-Method Explainability for YOLO26-Based Caries Detection in Intra-oral Photos

First Author Name[1], Second Author Name[2], Third Author Name[3],
Fourth Author Name[4], Fifth Author Name[5], Sixth Author Name[6]
[1,6]Department Name, Institution Name
City, Country
[2,3,4,5]Department Name, Institution Name
City, Country
[1]author1@example.com, [2]author2@example.com, [6]author6@example.com

***Abstract*—A dentist looking at an AI-flagged caries lesion has two reasonable questions: is the model right, and why does it think so? Most segmentation models that could answer the first question well are too heavy to run chairside and too opaque to answer the second one at all. We tackle both problems with a single pipeline built on YOLO26, trained to segment nine tooth pathology classes in intra-oral photographs from the AlphaDent dataset. Against a YOLOv8x baseline evaluated at 640 px and 960 px, we test three configurations: YOLO26m at 640 px and YOLO26l/YOLO26x at 960 px. The medium model turns out to be a remarkably good bargain. It reaches 0.4415 Box mAP@50 and 0.4298 Mask mAP@50 with 23.5M parameters and 121.2 GFLOPs, giving up only 1.65 mAP points to the 71.8M-parameter baseline while cutting compute by 64.8%. At 960 px, YOLO26l comes within 0.77 points of the baseline (0.4503), while the largest YOLO26x trails both, a reminder that model capacity should be matched to dataset size. Training scales across GPUs through Distributed Data Parallel (roughly 1.85× faster on two Tesla T4s), and a self-healing validation routine survives the CUDA out-of-memory crashes that high-resolution dental photographs otherwise cause. To answer the "why" question, our YOLOExplainer module hooks into intermediate YOLO26 layers and produces class activation maps with seven attribution methods, letting clinicians see, and cross-check, exactly which image regions drive each prediction.**

***Index Terms*—Dental caries detection, YOLO26, instance segmentation, explainable AI, class activation mapping, distributed data parallel, intra-oral photography**

I. Introduction

Dental caries is among the most common chronic diseases in the world, and much of its damage is preventable if lesions are caught early. Intra-oral photography is an appealing screening channel for exactly this reason: it is cheap, radiation-free, and increasingly available even outside specialist clinics. Deep detectors have already shown they can pull clinically useful signal from dental images, from tooth detection and numbering in panoramic radiographs [1] to interproximal caries in bitewings [2], pulp involvement in primary molars [3], and GV Black caries classification [4].

Yet two practical obstacles keep these models out of everyday practice. The first is cost: the best instance segmentation models are large, and a 70M-parameter network is a poor fit for a chairside tablet or an edge device in a rural clinic. The second is trust. A dentist cannot act on a prediction they cannot inspect, and the medical AI literature has been blunt about the need for explainability and causability before clinical adoption [5], [6].

We address both obstacles together. Starting from the reference YOLOv8 pipeline [7] for the AlphaDent dataset [8], a collection of intra-oral photographs annotated with nine tooth pathology classes, we migrate to the YOLO26 family of unified real-time vision models [9], whose `Segment26` head is well suited to fine-grained multi-class medical instance segmentation. Along the way we fix several engineering problems that quietly break reproducibility, and we add an explainability layer designed for clinical inspection rather than as an afterthought. Our contributions are:

- A YOLO26-based instance segmentation pipeline for nine-class dental pathology detection in intra-oral photographs, evaluated in three configurations (YOLO26m at 640 px, YOLO26l and YOLO26x at 960 px) against YOLOv8x baselines at 640 px and 960 px.
- Dynamic multi-GPU training via PyTorch Distributed Data Parallel (DDP), replacing the baseline's hard-coded single device, together with a self-healing validation routine that survives CUDA out-of-memory (OOM) events on very high-resolution images.
- *YOLOExplainer*, a self-contained module that produces class activation maps for YOLO26 predictions using seven complementary post-hoc methods, so that clinicians can visually audit which regions drive each pathology call.
- An accuracy–efficiency analysis showing that YOLO26m-seg keeps near-baseline accuracy at roughly one third of the parameters and 35.2% of the compute of YOLOv8x-seg.

II. Literature Review

Real-time detection and instance segmentation. Since the original YOLO reframed detection as a single-pass regression problem [10], the family has iterated quickly: YOLOv8 [7], YOLOv9 with programmable gradient information [11], the NMS-free end-to-end YOLOv10 [12], YOLO11 [13], the attention-centric YOLOv12 [14], and most recently YOLO26, which unifies detection, segmentation, pose, and classification

979-8-3195-1287-1/26/$31.00 ©2026 IEEE

in one end-to-end design [9]. On the segmentation side, Mask R-CNN [15] set the two-stage standard, while YOLACT [16] showed that prototype masks could deliver real-time instance segmentation, the idea modern YOLO segmentation heads still build on.

Deep learning in dental imaging. Convolutional networks reached expert-level tooth detection and numbering on panoramic radiographs years ago [1], and recent YOLOv8 studies have moved into genuinely clinical territory: interproximal caries in bitewings [2], pulp involvement in primary molars [3], and caries classification [4]. Progress, however, is bottlenecked by data. Uribe et al. survey the public dental dataset landscape and find it thin [17]. AlphaDent [8] helps by contributing densely annotated intra-oral photographs across nine pathology classes, and open frameworks such as SegmentAnyTooth target tooth enumeration and segmentation on the same modality [18].

Explainable AI for medical detection. Class activation mapping began with CAM [19] and has since branched into a family of methods with different strengths: Grad-CAM's gradient weighting [20], Grad-CAM++'s second-order refinement for multiple instances [21], the axiom-grounded XGrad-CAM [22], HiRes-CAM's faithfulness guarantee [23], Layer-CAM's use of intermediate layers [24], and the gradient-free Eigen-CAM [25]. Post-hoc CAM analysis has been applied to YOLO-based medical diagnosis before [26], but existing wrappers rarely speak to modern YOLO segmentation heads. Closing that gap for `Segment26` is one of the goals of this work.

III. Methodology

Fig. 1 gives an overview of the pipeline: automated dataset staging, DDP training, memory-safe validation, and multi-method XAI attribution. The subsections below walk through each stage and, where relevant, the failure it was designed around.

A. Dataset and Preprocessing

We use AlphaDent [8], a set of high-resolution intra-oral photographs annotated with polygon instance masks for nine dentist-defined pathology classes, caries among them. A small but instructive detail: symbolically linking the label directory from a read-only competition mount seems harmless, but YOLO writes `.cache` scan files next to the labels, so training crashes. The pipeline therefore copies images, labels, and the dataset YAML recursively into a writable working directory (`shutil.copytree(..., dirs_exist_ok=True)`), falling back to a Zenodo download when the competition mount is absent. We train at two input scales, 640×640 (YOLO26m) and 960×960 (YOLO26l, YOLO26x), matching the two baseline YOLOv8x resolutions.

B. YOLO26 Architecture

YOLO26 [9] is a unified, end-to-end real-time vision model family. We use its instance segmentation variants, whose `Segment26` head predicts prototype masks and per-instance

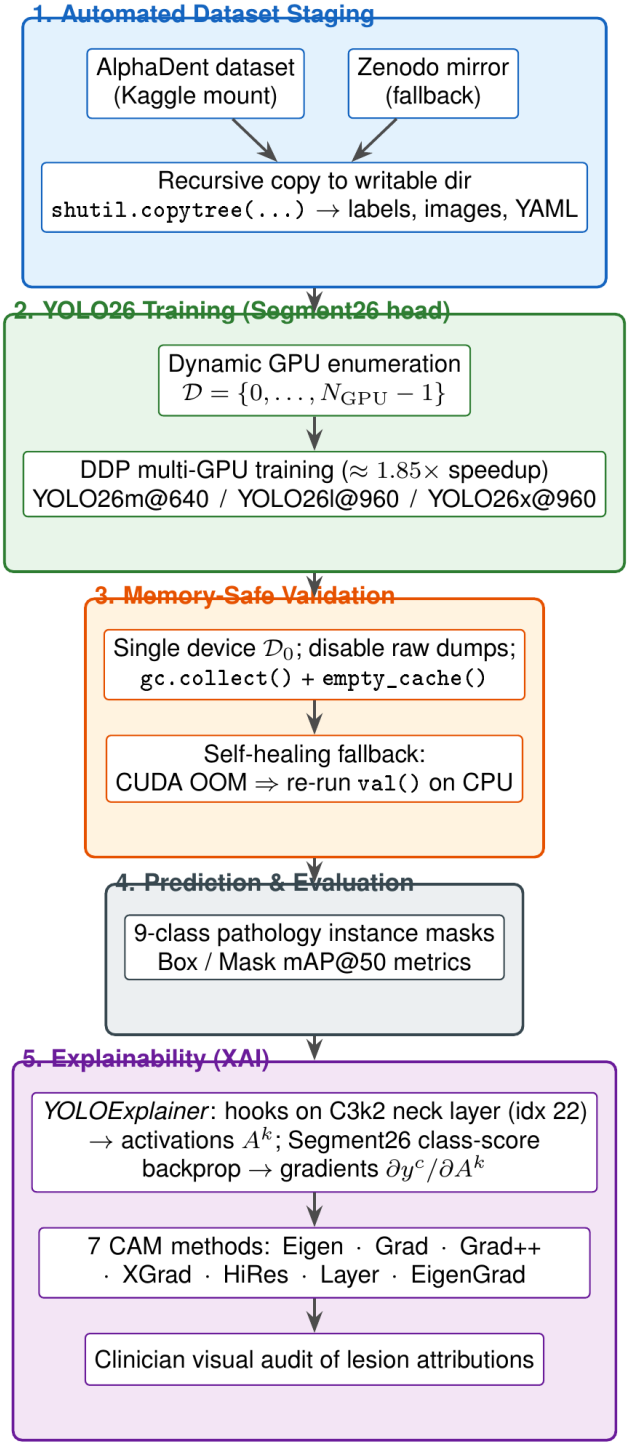

Fig. 1: Overview of the proposed interpretable YOLO26 pipeline for dental pathology instance segmentation.

coefficients in the YOLACT style [16], decoded jointly with box regression and nine-class logits. The size difference from the baseline is what makes chairside deployment plausible: YOLOv8x-seg carries 71.8M parameters and 344.1 GFLOPs, while YOLO26m-seg needs 23.5M parameters and 121.2 GFLOPs (fused), reductions of 67.2% and 64.8%.

C. Dynamic Multi-GPU Training with DDP

The baseline scripts hard-code `device=[0]`, so a second GPU sits idle unless the user edits the source. We instead enumerate whatever is available at runtime,

$$\mathcal{D} = \{0, 1, \ldots, N_{\text{GPU}} - 1\}, \tag{1}$$

and launch DDP training across all of it. Validation and prediction are deliberately routed back to a single device ($\mathcal{D}_0$): DDP validation loops are prone to multi-threaded metric-aggregation conflicts, and a single device sidesteps them entirely. On two Tesla T4 GPUs, 50 epochs finish in 1.46 hours, roughly a 1.85× speedup over the single-GPU configuration.

D. Self-Healing Memory-Safe Validation

This component exists because validation kept crashing. AlphaDent images are very large, and with `save_json` or `save_txt` enabled, YOLO reconstructs masks at full resolution via bilinear interpolation; on multi-instance images this transiently allocates up to 15.69 GiB and kills the GPU kernel. A naive retreat to the CPU fails differently, since holding all the scaled float tensors in system RAM invites the OS OOM killer instead. The routine we settled on does three things: it disables raw-prediction dumping so metrics are computed on downscaled mask grids, it flushes memory (`gc.collect()`, `torch.cuda.empty_cache()`) before evaluation, and it wraps the whole call in a fallback that catches CUDA OOM errors and re-runs on CPU, as summarized in Algorithm 1.

Algorithm 1 Self-Healing Memory-Safe Validation

```
Disable save_json, save_txt, save_conf
gc.collect(); torch.cuda.empty_cache()
try: metrics ← model.val(device = D_0, batch = 1, FP16)
except RuntimeError e:
if "out of memory" ∈ e then
    gc.collect(); torch.cuda.empty_cache()
    metrics ← model.val(device = CPU, batch = 1, FP32)
else
    raise e
end if
```

We also fixed a quiet but severe bug in the baseline CLI: `--conf` and `--iou` were declared as `type=int`, which truncates a threshold like 0.001 to 0. All thresholds in our pipeline are plain, type-safe Python floats.

E. Explainability: The YOLOExplainer Framework

A prediction a clinician cannot inspect is a prediction they cannot trust, so we built explainability into the pipeline rather than bolting it on. *YOLOExplainer* registers forward and backward hooks on a chosen intermediate YOLO26 layer, by default the final `C3k2` neck block (layer index 22), to capture feature activations A^k. It then back-propagates the summed per-class score from the `Segment26` head output to obtain gradients $\partial y^c / \partial A^k$, from which class activation maps are computed. If no target class is given, the module explains whichever of the nine pathology channels has the largest aggregate score. Generic CAM wrappers do not handle modern YOLO segmentation outputs; this one operates directly on `Segment26` predictions and supports seven attribution methods:

1) **Eigen-CAM** [25]: first principal component of the 2D activations; gradient-free and highly robust.
2) **Grad-CAM** [20]: activations weighted by spatially averaged class-score gradients.
3) **Grad-CAM++** [21]: second-order gradient weighting, which helps when several lesions co-occur.
4) **XGrad-CAM** [22]: gradients scaled by normalized activations to satisfy sensitivity and conservation axioms.
5) **HiRes-CAM** [23]: element-wise activation–gradient product, with a guarantee of faithfulness to the decision path.
6) **Layer-CAM** [24]: positive-gradient spatial weighting of intermediate layers, useful for fine detail.
7) **EigenGrad-CAM**: first principal component of the gradient–activation product, pairing Eigen-CAM's robustness with class discrimination.

Why seven methods instead of one? Because each has known failure modes, and showing them side by side (Fig. 2) lets a clinician check whether the attributions agree before trusting any of them, which is much closer to how the causability literature says medical explanations should work [5], [26].

Fig. 2: Class activation maps produced by the seven XAI methods for a caries instance predicted by YOLO26 on an AlphaDent intra-oral photograph.

IV. Results and Discussions

A. Experimental Setup

All models are trained on AlphaDent with the Ultralytics framework for 100 epochs, using DDP across two Tesla T4 GPUs with default Ultralytics segmentation hyperparameters and augmentation; YOLO26m uses batch size 16 at 640 px, while YOLO26l and YOLO26x use batch size 8 at 960 px. Validation runs at batch size 1 with FP16 inference on GPU (full precision on the CPU fallback), a confidence threshold of 0.001, and an IoU threshold of 0.5; for qualitative figures we raise the confidence threshold to 0.25 for visual clarity. The baseline YOLOv8x-seg is evaluated at 640 px and 960 px; our YOLO26m-seg is trained at 640 px and YOLO26l-seg/YOLO26x-seg at 960 px. Table I compares model complexity and Table II reports accuracy. Cells marked "–" correspond to baseline results not published in usable form by the reference pipeline.

TABLE I: Model Complexity Comparison

Model	Input (px)	Params (M)	GFLOPs
YOLOv8x-seg (baseline) [7]	640 / 960	71.8	344.1
YOLO26m-seg (ours)	640	**23.5**	**121.2**
YOLO26l-seg (ours)	960	27.9	139.4
YOLO26x-seg (ours)	960	62.7	313.0
Reduction (26m vs. v8x)		−67.2%	−64.8%

TABLE II: Instance Segmentation Results on AlphaDent (100 Epochs)

Model	Input (px)	Box mAP@50	Mask mAP@50
YOLOv8x-seg (baseline)	640	–	–
YOLOv8x-seg (baseline)	960	0.4580	–
YOLO26m-seg (ours)	640	0.4415	0.4298
YOLO26l-seg (ours)	960	**0.4503**	**0.4392**
YOLO26x-seg (ours)	960	0.4128	0.3972

B. Accuracy–Efficiency Trade-off

The headline number is how little accuracy the smaller models give up. YOLO26m-seg reaches 0.4415 Box mAP@50 and 0.4298 Mask mAP@50, sitting 1.65 points below the YOLOv8x baseline's 0.4580 while using about a third of the parameters and 35.2% of the compute. For a screening tool that has to run on modest hardware, that is a trade most deployments would take without hesitation. At 960 px, YOLO26l-seg comes closest to the baseline, reaching 0.4503 Box mAP@50 and the best Mask mAP@50 of the family (0.4392) with 27.9M parameters and 139.4 GFLOPs, i.e. 61.1% fewer parameters and 59.5% less compute than YOLOv8x, while sitting only 0.77 box points below the baseline at the same resolution. Interestingly, scaling further does not help: YOLO26x-seg at 960 px lands at 0.4128/0.3972, below both YOLO26l at the same resolution and YOLO26m at 640 px. On a dataset of AlphaDent's size, with several rare pathology classes, the largest model appears to overfit rather than benefit from added capacity, a pattern consistent with its higher validation precision but markedly lower recall (0.55/0.42 vs. 0.44/0.47 for YOLO26m). The pipeline also reports structured Mask mAP@50 benchmarks, which the baseline CLI never published in usable form and which matter for pixel-accurate lesion delineation. Qualitative predictions across the nine pathology classes are shown in Fig. 3.

Fig. 3: Qualitative instance segmentation results of the proposed YOLO26 models on AlphaDent test images (nine pathology classes).

C. Training Scalability and Robustness

DDP scaling behaved close to linearly in practice: 50 epochs on two T4 GPUs took 1.46 hours, about 1.85× faster than the single-GPU setup. The memory-safe validation routine did the job it was built for. The transient 15.69 GiB allocations that used to abort high-resolution evaluation simply no longer occur, and on the rare occasion an OOM does slip through, the CPU fallback absorbs it and the run finishes rather than dies.

D. Interpretability Analysis

Looking across the seven CAM methods, the attributions for correctly detected caries instances concentrate where a dentist would want them to: on the lesion surface and the adjacent enamel margins. The methods have distinct personalities. Eigen-CAM, being gradient-free, stays the most stable under the specular reflections that plague intra-oral photography; HiRes-CAM and Layer-CAM recover the sharpest lesion boundaries; Grad-CAM++ does the best job of separating several lesions on one tooth. Agreement across methods also turned out to be a useful confidence signal in itself. Predictions on which all seven maps coincided spatially were, qualitatively, the more reliable ones, which suggests multi-method XAI can serve as a lightweight clinical audit tool [6].

V. Discussion

The main takeaway is that the price of shrinking from a 71.8M-parameter YOLOv8x to a 23.5M-parameter YOLO26m is small, 1.65 mAP points against a 64.8% compute saving, which makes chairside and mobile deployment realistic without server-class hardware. It is worth being honest about what

the engineering contributions are: DDP orchestration, automated dataset staging, type-safe thresholds, and OOM-resilient validation do not make the model smarter, but they remove exactly the kind of reproducibility failures that stall clinical ML pipelines in practice. A second takeaway is that capacity must match data: at 960 px the large YOLO26l narrowly trails the baseline (0.4503 vs. 0.4580), while the extra-large YOLO26x underperforms both YOLO26l and the far cheaper YOLO26m, suggesting that on a single moderately sized clinical dataset the biggest model is the wrong choice, not just the most expensive one. Limitations remain. Evaluation is on a single dataset, the 640 px baseline run is not published, and our XAI assessment is qualitative; deletion/insertion faithfulness curves and reader studies with practicing dentists are the obvious next steps.

VI. Conclusion

We set out to build a caries detector a clinic could actually run and a clinician could actually question, and the pieces are now in place: a YOLO26 pipeline that keeps near-baseline accuracy (0.4415 vs. 0.4580 Box mAP@50) at a third of the size, near-linear multi-GPU training via DDP, validation that heals itself instead of crashing, and a seven-method CAM framework wired to the YOLO26 segmentation outputs. Across the family, YOLO26l at 960 px comes nearest the baseline (0.4503 Box mAP@50) while YOLO26x overfits, confirming that model scale should be chosen against dataset size rather than maximized. Future work will extend evaluation to external intra-oral and radiographic datasets [17] and put the explanations in front of dentists to quantify their faithfulness and clinical value.

References

[1] D. V. Tuzoff, L. N. Tuzova, M. M. Bornstein, A. S. Krasnov, M. A. Kharchenko, S. I. Nikolenko, M. M. Sveshnikov, and G. B. Bednenko, "Tooth detection and numbering in panoramic radiographs using convolutional neural networks," *Dentomaxillofacial Radiology*, vol. 48, no. 4, p. 20180051, 2019.

[2] M. Bayati *et al.*, "Advanced ai-driven detection of interproximal caries in bitewing radiographs using yolov8," *Scientific Reports*, vol. 15, p. 4641, 2025.

[3] Sohrabi *et al.*, "Deep learning-based assessment of pulp involvement in primary molars using yolov8," *PLOS Digital Health*, vol. 4, no. 4, p. e0000816, 2025.

[4] R. Z. Agulan, J. V. Magsumbol, and M. A. Rosales, "Detection of dental caries based on gv black classification utilizing yolov8 from x-ray images," in *IEEE International Conference on Imaging Systems and Techniques (IST)*, 2024.

[5] A. Holzinger, G. Langs, H. Denk, K. Zatloukal, and H. Müller, "Causability and explainability of artificial intelligence in medicine," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 9, no. 4, 2019.

[6] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on explainable artificial intelligence (xai)," *IEEE Access*, vol. 6, pp. 52 138–52 160, 2018.

[7] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[8] E. I. Sosnin, Y. L. Vasilev, R. A. Solovyev, A. L. Stempkovskiy, D. V. Telpukhov, A. A. Vasilev, A. A. Amerikanov, and A. Y. Romanov, "Alphadent: A dataset for automated tooth pathology detection," p. 6, 2025.

[9] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, "Ultralytics yolo26: Unified real-time end-to-end vision models," *arXiv preprint arXiv:2606.03748*, 2026.

[10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[11] C.-Y. Wang, I.-H. Yeh, and H. Liao, "Yolov9: Learning what you want to learn using programmable gradient information," *arXiv preprint arXiv:2402.13616*, 2024.

[12] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "Yolov10: Real-time end-to-end object detection," *arXiv preprint arXiv:2405.14458*, 2024.

[13] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics yolo11," https://github.com/ultralytics/ultralytics, 2024.

[14] Y. Tian, Q. Ye, and D. Doermann, "Yolov12: Attention-centric real-time object detectors," *arXiv preprint arXiv:2502.12524*, 2025.

[15] K. He, G. Gkioxari, P. Dollár, and R. B. Girshick, "Mask R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[16] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, "YOLACT: Real-time instance segmentation," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2019.

[17] S. E. Uribe, J. Issa, F. Sohrabniya, A. Denny, N. N. Kim, A. F. Dayo, A. Chaurasia, A. Sofi-Mahmudi, M. Büttner, and F. Schwendicke, "Publicly available dental image datasets for artificial intelligence," *Journal of Dental Research*, vol. 103, no. 13, pp. 1365–1374, 2024.

[18] K. D. Nguyen, H. T. Hoang, T.-P. H. Doan, K. Q. Dao, D.-H. Wang, and M.-L. Hsu, "Segmentanytooth: An open-source deep learning framework for tooth enumeration and segmentation in intraoral photos," *Journal of Dental Sciences*, 2025.

[19] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning deep features for discriminative localization," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2921–2929.

[20] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626.

[21] A. Chattopadhay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, "Grad-cam++: Generalized gradient-based visual explanations for deep convolutional networks," in *IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2018, pp. 839–847.

[22] R. Fu, Q. Hu, X. Dong, Y. Guo, Y. Gao, and B. Li, "Axiom-based grad-cam: Towards accurate visualization and explanation of cnns," in *Proceedings of the British Machine Vision Conference (BMVC)*, 2020.

[23] R. L. Draelos and L. Carin, "Use hirescam instead of grad-cam for faithful explanations of convolutional neural networks," *arXiv preprint arXiv:2011.08891*, 2020.

[24] P.-T. Jiang, C.-B. Zhang, Q. Hou, M.-M. Cheng, and Y. Wei, "Layercam: Exploring hierarchical class activation maps for localization," *IEEE Transactions on Image Processing*, vol. 30, pp. 5875–5888, 2021.

[25] M. B. Muhammad and M. Yeasin, "Eigen-cam: Class activation map using principal components," *arXiv preprint arXiv:2008.00299*, 2020.

[26] P. P. S. Borah, D. Kashyap, R. A. Laskar, and A. J. Sarmah, "A comprehensive study on explainable ai using yolo and post hoc method on medical diagnosis," *Journal of Physics: Conference Series*, vol. 2919, no. 1, p. 012045, 2024.